

Lecture 9B-10A Machine Learning Algorithms I

(Decision Tree and Single Linear Regression Analysis)

Algorithm # 1: Decision Tree – a recursive algorithm builds one node at a time

Algorithm DecisionTree;

Input: 1. Attribute matrix S^M ;

% S^M is a matrix where each row is an attribute and each column is a value of the attribute

% S^A is a set of the form $\{(\text{attribute}, \text{set of triples (attribute-value, +ve outcomes, -ve outcomes)})\dots\}$

Output: 1. A decision tree T ;

{ $S^A = \text{convert_matrix_to_set}(S^M)$; $T = \text{build_decision_tree}(S^A)$; **return**(T); }

Algorithm build_decision_tree

Input: 1. S^A – set of the form $\{(\text{attribute}, \text{set of triples (attribute-value, +ve outcomes, -ve outcomes)})\dots\}$

Output: 1. Tree T ;

```
{
    decision tree  $T = \text{null}$ ; % initialize the tree to null tree
    current attributes set  $S^{\text{cur}} = S^A$ ;
    Initial information  $\text{inf} = 1$ ;
    The set of entropy-attribute pairs  $S^E = \forall s \in S^{\text{cur}} \{ (\pi_1(s), \text{compute\_entropy}(\pi_2(s))), \dots \}$ 
    The set of information change  $S^{\Delta E} = \forall \text{eap} \in S^E \{ (\text{inf} - \pi_2(\text{eap})), \dots \}$ 
    % find out the attribute-entropy change pair with maximum information change
    next attribute-entropy pair  $\text{ae} = \text{maximum}(S^{\Delta E})$ ;
    attribute  $a = \pi_1(\text{ae})$ ; value  $v = \pi_2(\text{ae})$ ; % get attribute and the corresponding entropy
    most entropy change attribute-values  $(\text{attr}, S^{\text{val}}) = \text{get\_attribute}(\text{ae}, S^{\text{cur}})$ 
     $S^B = \forall v \in S^{\text{val}} \{ (\pi_1(v), \text{compute\_entropy}(\pi_2(v), \pi_3(v))), \dots \}$ ;
     $S^{\text{cur}} = S^{\text{cur}} - (\text{attr}, v)$ ; % remove processed attribute from further consideration
    % recursive build decision tree for remaining attributes
    %  $S^{\text{subtree}}$  is the set of subtrees for each leaf node value
     $S^{\text{subtree}} = \forall \text{leaf-node } l \in S^B \{ \text{build\_decision\_tree}(S^{\text{cur}}) \}$ 
     $T = \text{insert\_branches}(\text{attr}, v, S^{\text{subtree}})$ ; % insert the branch for each value of the attribute
    return  $T$ ;
}
```

Algorithm #2: Linear regression analysis

Algorithm regression_analysis

Input: 1. number of points n ; 2. sequence of points $S = \langle (x_1, y_1), \dots, (x_n, y_n) \rangle$;

Output: 1. A pair (slope, intercept);

N = read from the terminal;

S = read from the terminal;

$\{i = 1; x_total = \sum_{i=1}^{i=n} x_i; y_total = \sum_{i=1}^{i=n} y_i$

$x_avg = x_total/n; y_avg = y_total / n;$

$variance_x = \frac{1}{(n-1)} \sum_{i=1}^{i=n} (x_i - x_avg)^2$; $covariance_xy = \frac{1}{(n-1)} \sum_{i=1}^{i=n} (x_i - x_avg) \times (y_i - y_avg)$

$slope = covariance_xy / variance_x;$

$intercept = y_avg - slope \times x_avg;$

return (slope, intercept);

}

Lectures 10B-11A Clustering Algorithms

This tutorial contains elaborate algorithm for four algorithms: 1) K-means clustering; 2) Hierarchical clustering; 3) Density-based clustering; 4) Graph-based clustering. It does not include incremental K-means clustering which I will leave you as an exercise to practice.

These are by no means ideal algorithms. However, they teach you art of writing algorithms using the integration of abstractions like sets, sequences (ordered sets), universal and existential quantifiers, and control abstractions such as if-then-else, while-loop and recursion.

Note that arrays, linked lists, vectors are all implementation techniques for sequences which are themselves ordered sets. Similarly, for-loop can be easily modeled using universal quantifiers in many cases.

An algorithm mainly contains input and output. Sometimes, you can also use global variables to pass the affect to the calling algorithm. Unlike processes, algorithm always terminates.

In the algorithm given in the following pages, following convention has been used:

- (a) Reserved words for control abstraction have been written in bold to show the control flow
- (b) Comments have been describe by “%” followed by comments. Comments are also in blue colored text to separate from program statements
- (c) Program statements are written in black.
- (d) Each algorithm is numbered such as Algorithm # 1, Algorithm # 2, Algorithm #3, Algorithm # 4.

Algorithm # 1: K-means clustering

Algorithm k_means_clustering

Input: 1. A sequence of points S^P read from a file; 2. Number of clusters n^{cluster} read from the terminal;

3. Number of maximum iterations **max** read from the terminal;

4. User defined maximum allowable shift of the centroid in next cycle ε

Output: 1. A sequence of pairs: (centroid, set of points in a cluster);

{ read the points from a file F in a sequence of pairs $S^P = \langle (x_1, y_1), \dots, (x_n, y_n) \rangle$;

n^{cluster} = read from the terminal number of clusters;

ε = read from the terminal;

$n^P = \text{length}(S^P)$; % find the number of points in the set of points

max = read from the terminal maximum number of loops;

$(S^{\text{seed}}, \text{radius}) = \text{generate_seed_points}(S^P, n^{\text{cluster}})$;

count = 1; % initialize the counter used to terminate after maximum number of iterations

% the Boolean variable **stabilized** is used to check for terminating the execution

% The variable stabilized becomes true when centroids of clusters do not move beyond a threshold

stabilized = false;

while ((count < max) $\wedge$ $\neg$ stabilized) { % Clustering loop

$S^{\text{out}} = S^P$; % initialize set of outliers S^{out} to all the points in the set S^P

forall ($1 \leq i \leq n^{\text{cluster}}$) { % go through the new cycle of computing clusters and centroids

forall ($1 \leq j \leq n^P$) {

$d_{ij} = \sqrt{(cx_i - x_j)^2 + (cy_i - y_j)^2}$; % Euclidean distance between the centroid and point

if ($d_{ij} < \text{radius}$) {

cluster_i = (x_j, y_j) + cluster_i; % insert point (x_j, y_j) in the cluster

outliers = outliers - (x_j, y_j) ; % remove the point (x_j, y_j) from outliers;

}

}

% compute coordinates of the new centroid of cluster_i

$$Cx_{i_new} = \frac{\sum_{j=1}^{\text{size}(\text{cluster}_i)} \text{cluster}_i.x}{\text{size}(\text{cluster}_i)} ; \quad Cy_{i_new} = \frac{\sum_{j=1}^{\text{size}(\text{cluster}_i)} \text{cluster}_i.y}{\text{size}(\text{cluster}_i)}$$

print("cluster #", i, new centroid: (Cx_{i_new}, Cy_{i_new}));

}

Print("Current outliers", outliers); stabilized = true;

forall ($1 \leq i \leq n^c$) { % for every cluster find out the centroid shift

centroid_shift = $\sqrt{(cx_i - cx_{i_new})^2 + (cy_i - cy_{i_new})^2}$ % calculate centroid shift

if (centroid_shift > ε) stabilized = false; % if centroid shift ε then clusters are unstable

}

count = count + 1; % go to the next iterative loop

}

Print("Final Clusters and Outliers"); **forall** ($1 \leq i \leq n^c$) display(cluster_i); display(outliers);

}

Algorithm generate_seed_points

Input: 1. Sequence of points S^P ; 2. number of clusters n^{cluster} ;

Output: Set of seed points and radius S^T ;

% S^x is the set of x-coordinate values of the points in S^P ;

% S^y is the set of y-coordinate values of the points in S^P ;

{ $S^x = \Pi_1(S^P)$; $S^y = \Pi_2(S^P)$; % get the set of x-coordinates and the set of y-coordinates

total points $n^P = \text{length}(S^P)$; % length gives the number of elements

$x^{\max} = \text{maximum}(S^x)$; $x^{\min} = \text{minimum}(S^x)$; $y^{\max} = \text{maximum}(S^y)$; $y^{\min} = \text{minimum}(S^y)$;

size_x = $(x^{\max} - x^{\min}) / n^{\text{cluster}}$; size_y = $(y^{\max} - y^{\min}) / n^{\text{cluster}}$;

macroblock numbers $n^{\text{macro}} = n^{\text{cluster}} \times n^{\text{cluster}}$; Average density $\delta = |S^P| / n^{\text{macro}}$

initialize set of macroblocks with higher density $S^{\text{higher}} = \{\}$;

Vi (0 ≤ i < n^{cluster}) { % for every macro block x-division do

% calculate the start and end of x-coordinates for the macroblock

$x^{\text{low}} = \min_x + i \times \text{size_x}$; $x^{\text{high}} = x^{\text{low}} + \text{size_x}$;

$x^{\text{mid}} = (x^{\text{low}} + x^{\text{high}}) / 2$; % calculate the middle x-coordinates to find the centroid

Vj (0 ≤ j < n^{cluster}) { % for each y division of a macro block do

% calculate the start and end of y-coordinates for the macroblock

$y^{\text{low}} = \min_y + j \times \text{size_y}$; $y^{\text{high}} = y^{\text{low}} + \text{size_y}$;

$y^{\text{mid}} = (y^{\text{low}} + y^{\text{high}}) / 2$; calculate the middle y-coordinates to find the centroid

$n^P = \text{points_in_macroblocks}(S^P, x^{\text{low}}, y^{\text{low}}, x^{\text{high}}, y^{\text{high}})$;

% if points > density then mid-point becomes a seed point

if ($n^P > \delta$) $S^{\text{higher}} = S^{\text{higher}} + (x^{\text{mid}}, y^{\text{mid}})$;

}

}

initialize $S^{\text{seed}} = \{\}$; % initialize the set of final seed points S^{seed} to an empty set

Vi (1 ≤ i ≤ n^{cluster}) { % randomly select seed points

next_seed = randomly_select(S^{higher}); % randomly select one of the seed points

$S^{\text{seed}} = S^{\text{seed}} + \text{next_seed}$; % put the seed point in the set of selected seeds S^{seed}

$S^{\text{higher}} = S^{\text{higher}} - \text{next_seed}$; % remove the selected seed from the set S^{higher}

}

radius = minimum(size_x, size_y) / n^{cluster} ; % Initialize radius

Vi (1 ≤ i ≤ n^{cluster}) { % for every cluster repeat the following process

(x_i, y_i) = ith_element(S^{seed}, i); % extract the coordinates of the ith seed

Vj (1 ≤ j ≤ n^{cluster}) { % compare the distance between two clusters pairwise

if (i ≠ j) {

(x_j, y_j) = ith_element(S^{seed}, j); % extract the coordinates of the jth seed

$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$; % find the distance between two cluster-centroids

% radius is set to half the distance between two smallest clusters

if ($d_{ij} < 2 * \text{radius}$) radius = $d_{ij} / 2$;

}

}

return ($S^{\text{seed}}, \text{radius}$);

}

}

Algorithm points_in_macroblock

% algorithm counts number of points in the given macroblock

Input: 1. Set of points S^p of the form $\{(x_1, y_1), \dots\}$

2. Left bottom x-coordinates X^{low} ; left bottom y-coordinate Y^{low} ;

3. Right top x-coordinates X^{top} and top coordinates Y^{top} ;

Output: 1. number of points in the macroblock n^{macro} ;

{ initialize $n^{\text{macro}} = 0$;

$\forall (x, y) \in S^p$ { if $((x \geq X^{\text{low}}) \wedge (x < X^{\text{top}}) \wedge (y \geq Y^{\text{low}}) \wedge (y < Y^{\text{top}}))$ $n^{\text{macro}} = n^{\text{macro}} + 1$;

return n^{macro} ;

}

Algorithm #2: Hierarchical Clustering

Algorithm Hierarchical Clustering

Input: 1. Set of leaf-level clusters S^{leaf} of the form (label, centroid, count) % single point has count = 1

Output: A Tree T;

```
{
     $S^{\text{leaf}}$  = read the data from the terminal or a file;
    % each leaf point could be a single point or a K-means cluster
    % each leaf is modeled as a cluster of the form (label, centroid (x, y), count);
     $S^{\text{dist}} = \forall l_1, l_2 \in S^{\text{leaf}}$  such that  $l_1 \neq l_2 \{ (l_1.\text{label}, l_2.\text{label}, \text{euclidean\_distance}(l_1.\text{centroid}, l_2.\text{centroid})), \dots \}$ 
     $S^{\text{cur}} = \text{sort\_on\_distance}(S^{\text{dist}})$ ;  $S^{\text{label}} = \forall s \in S^{\text{leaf}} \{s.\text{label}, \dots\}$ ;
     $T = \forall n \in S^{\text{leaf}} \{ (\Lambda, n, \Lambda), \dots \}$  % insert leaf nodes in the tree where  $\Lambda$  shows null pointers
    while (size( $S^{\text{cur}}$ ) > 1) {
        next pair p = remove_first( $S^{\text{cur}}$ ); % remove the shortest distance pair from the ordered set  $S^{\text{cur}}$ 

        fused centroid c = 
$$\begin{pmatrix} \frac{\pi_1(p).\text{centroid}.x \times \pi_1(p).\text{count} + \pi_2(p).\text{centroid}.x \times \pi_2(p).\text{count}}{\pi_1(p).\text{count} + \pi_2(p).\text{count}}, \\ \frac{\pi_1(p).\text{centroid}.y \times \pi_1(p).\text{count} + \pi_2(p).\text{centroid}.y \times \pi_2(p).\text{count}}{\pi_1(p).\text{count} + \pi_2(p).\text{count}} \end{pmatrix};$$


        newLabel =  $l_1.\text{label} \cup l_2.\text{label}$ ; % parent label is union of set of children labels
        % create a new parent node
        create parent node par = ( $l_1.\text{label}$ , newLabel, c, ( $l_1.\text{count} + l_2.\text{count}$ ),  $l_2.\text{label}$ );
        T = insert_node(T, par); % insert the new parent node in the tree
         $S^{\text{cur}} = \text{remove\_pairs}(l_1.\text{label}, S^{\text{cur}})$ ; % remove all the pairs having  $l_1.\text{label}$  as one of the nodes
         $S^{\text{cur}} = \text{remove\_pairs}(l_2.\text{label}, S^{\text{cur}})$ ; % remove all the pairs having  $l_2.\text{label}$  as a node
        % remove the labels of two leaf labels just fused and insert the label of the front node
         $S^{\text{newD}} = \forall s \in S^{\text{cur}} \{ (\text{par}.\text{label}, s.\text{label}, \text{distance}(c, s.\text{centroid})), \dots \}$ 
         $S^{\text{cur}} = \text{sort\_on\_distance}(S^{\text{cur}} \cup S^{\text{newD}})$ ; % add new distances from parent node and sort
    }
    return T;
}
```

Algorithm remove_pairs

Input: 1. Current node n; 2. Set of current distance-pairs fringe S^{fringe} ;

Output: Remaining set of distance-pairs fringe S^{new} ;

```
{  $S^{\text{new}} = \{ \}$ ;
    while is_not_empty( $S^{\text{fringe}}$ ) {
        s = remove_first( $S^{\text{fringe}}$ ); % extract the next node from set of distance pairs
        % collect on those distance pairs with both nomatching labels with the given node
        if (( $\pi_1(s).\text{label} \neq n.\text{label}$ )  $\wedge$  ( $\pi_2(s).\text{label} \neq n.\text{label}$ ))  $S^{\text{new}} = S^{\text{new}} \cup s$ ;
    }
    return  $S^{\text{new}}$ ;
}
```

Algorithm #3: Density based Clustering

% density based clustering is used to find clusters with irregular shape
% such as English letters, human organs in X-ray, object detection
% in satellite images, etc.

Algorithm density clustering;

% spatial region r is a rectangular block modeled as triple (coordinate of left bottom, width, height)

Input: 1. A spatial region R of the form (left-bottom, length, width); 2. Set of coordinates S^P ;

3. Density threshold τ ; 4. Size of macroblock s ;

Output: a set S^{cluster} of macroblock sets with density $> \tau$;

```
{  R = read a triple from the terminal;
    SP = read set of pairs from a file;
     $\tau$  = read from the terminal; s = read from the terminal;
    % Smacro is modeled as a 4-tuple (macroblock index, (Xlb, Ylb), size s, number of points)
    % (Xlb, Ylb) is the coordinate of the left bottom end of the macro block
    % together (Xlb, Ylb) and (Xlb + s, Ylb + s) define the macro block region
    set of macroblocks Smacro = create_macroblocks(R, SP, s); % create a set of macroblocks
    % Scluster is the set of density clusters in the region R
    % each element of Scluster is a set of macroblocks
    Scluster = {}; % initialize the density cluster Scluster to an empty set
    % map points to corresponding macroblocks index and store the indices in the set Smap
    Smap =  $\forall$  coordinates (x, y)  $\in S^P$  { map_to_macroblock((x, y), ...);
    Scur = Smacro; % initialize Scur to the set of macroblocks Smacro
    while is_not_empty(Smap) % compute and update the number of points in macroblocks
    {
        macroblock index i = remove_first(Smap); % remove the next macroblock index from Smap
        increment  $\pi_4(S^{\text{cur}}_i)$  by 1; % increment the number of points in macroblock by one
    }
    % Svisited is used to avoid cycles when finding adjacent macroblocks
    Svisited = {}; % initialize Svisited to an empty set
    while is_not_empty(Scur) % find the dense macroblocks above a density threshold
    {
        macro_block m = remove_first(Scur);
        if ( $\pi_4(m) > \tau$ ) % check for higher density block
        {
            SnewC = {m}; % initialize the new cluster SnewC as {m}
            % function expand_cluster recursively expands the fringe of Snew
            expand_cluster(SnewC, m); Scluster = Scluster + SnewC;
        }
    }
    return Scluster; % return the set of valid density clusters
}
```

Algorithm expand_clusters

% The recursive function finds all the macroblocks in a density cluster

Input: 1. Current cluster S^{newC} ; 2. Current macroblock m ;

Global: 1. Set of macroblocks S^{cur} ; 2. Set of visited nodes $S^{visited}$;

```
{
     $S^{cur} = S^{cur} - m$ ; % remove the macroblock m from set of unprocessed macroblocks
     $S^{adjacent} = \text{get\_adjacent\_nodes}(m)$ ; % the set  $S^{adjacent}$  is the set of adjacent nodes of m
    while is_not_empty( $S^{adjacent}$ ) % expand each adjacent node if it has higher density
    {
         $p = \text{remove\_first}(S^{adjacent})$ ; % p is the next adjacent node
        if ( $(p \notin S^{visited}) \wedge (\pi_4(p) > \tau)$ ) % if the point p is not already visited and has higher density
             $S^{newC} = S^{newC} \cup \text{expand\_clusters}(\{p\}, p)$ ; % recursive call to expand the fringe
    }
    return  $S^{newC}$ ;
}
```

Algorithm create_macro_blocks;

Input: 1. region R ; 2. set of point coordinates S^P ; 3. Size s ;

Output: set of macroblocks S^{macro} ;

% R is a tuple of the form $(X^{lb}, Y^{lb}, X^{rt}, Y^{rt})$;

% S^P is a sequence of the form $\{(x_i, y_i), \dots\}$

% S^{macro} has 3 fields: $(x^{lb}, y^{lb}, \text{number of points initialized to } 0)$

```
{
     $S^{macro} = \{\}$ ; % initialize set of macroblocks to an empty set
    Macroblock index  $i = 1$ ; % initialize macroblock index to one
     $start^y = Y^{lb}$ ; % initialize the Y-coordinate to the bottom most Y-coordinate
    while ( $start^y \leq Y^{rt}$ ) { % process one row at a time
         $start^x = X^{lb}$ ; % start from the leftmost macroblock of the current row
        while ( $start^x \leq X^{rt}$ ) % check if it is the last macroblock in the current row
        {
            % insert a new macroblock in the set  $S^{macro}$ 
             $S^{macro} = S^{macro} + (i, (start^x, start^y), (start^x + s, start^y + s), 0)$ ;
             $start^x = start^x + s$ ; % go to the next macroblock
             $i = i + 1$ ;
        }
         $start^y = start^y + s$ ; % go to the next row of macroblocks
    }
    return  $S^{macro}$ ; % return the set of macroblocks
}
```

Algorithm map_to_macro_block

% the algorithm maps a point to the index of the corresponding macroblocks

% leftmost bottommost macroblock in the region R is mapped to 1

% the index increases one row at a time from leftmost bottom

Input: 1. point coordinates p of the form (x, y) ;

2. Set of macroBlocks S^{macro} ; 3. Macroblock size s ;

4. region start $(X^{\text{lbR}}, Y^{\text{lbR}})$; 5. region end $(X^{\text{rtR}}, Y^{\text{rtR}})$;

Output: macroblock m 's index i ;

```
{
    blocks in a row  $b = \lceil (Y^{\text{rtR}} - Y^{\text{lbR}})/s \rceil$ ; % compute the number of blocks in a row;
     $\text{row}^{\text{block}} = \text{integer}((p.x - X^{\text{lb}})/s) + 1$ ;  $\text{col}^{\text{block}} = \text{integer}(p.y - Y^{\text{lb}})/s + 1$ ;
    index  $i = (\text{row}^{\text{block}} - 1) \times b + \text{col}^{\text{block}}$ ; % compute the index of the macroblock where point  $p$  resides
    return  $i$ ; % return the index of the macroblock where the point is located
}
```

Algorithm get_adjacent_macroblocks

Input: 1. Current macroblock m ;

2. Set of macroblocks S^{macro} ;

3. size of the macroblock s ;

4. region R coordinates $((X^{\text{lbR}}, Y^{\text{lbR}}), (X^{\text{rtR}}, Y^{\text{rtR}}))$;

Output: 1. A set of macroblocks S^{adj} ;

% a maximum of eight adjacent blocks can be there except on the edges or corners of the region

```
{
    blocks in a row  $b = \lceil (Y^{\text{rtR}} - Y^{\text{lbR}})/s \rceil$ ; % compute the number of blocks in a row
    % compute the row and column of the macroblock in the region
     $\text{Row}^{\text{block}} = \text{integer}((\pi_2(m).x - X^{\text{lbR}})/s) + 1$ ;  $\text{Col}^{\text{block}} = \text{integer}(\pi_2(m).y - Y^{\text{lbR}})/s + 1$ ;
    macroblock  $m$ 's index  $i = (\text{Row}^{\text{block}} - 1) \times b + \text{Col}^{\text{block}}$ ; % compute the index of the macroblock  $m$ 
     $S^{\text{index}} = \{\}$ ; % initialize the set of macroblock indices to an empty set
    if  $(\pi_2(m).x \geq X^{\text{lbR}} + s)$  % if the x coordinate of the macroblock start > region start + s
    {
         $S^{\text{index}} = S^{\text{index}} + (i - 1)$ ; % insert the previous block in the set of adjacent blocks
        % insert adjacent blocks in previous row
        if  $(\pi_2(m).y \geq Y^{\text{lbR}} + s)$   $S^{\text{index}} = S^{\text{index}} \cup \{(i - 1 - b), (i - b)\}$ ;
        % insert adjacent blocks in the following row to the set
        if  $(\pi_2(m).y < Y^{\text{rt}} - s)$   $S^{\text{index}} = S^{\text{index}} \cup \{(i + b), (i - 1 + b)\}$ ;
    }
    if  $(\pi_2(m).x < X^{\text{rt}} - s)$  % if the x coordinate of the macroblock start < region end - s
    {
         $S^{\text{index}} = S^{\text{index}} + (i + 1)$ ; % insert the right side adjacent macroblock in the set
        if  $(\pi_2(m).y \geq Y^{\text{lb}} + s)$   $S^{\text{index}} = S^{\text{index}} + (i + 1 - b)$ ; % insert lower row right side macroblock
        if  $(\pi_2(m).y < Y^{\text{rt}} - s)$   $S^{\text{index}} = S^{\text{index}} + (i + 1 + b)$  % insert right side upper row block
    }
     $S^{\text{adj}} = \forall \text{index } i^{\text{adj}} \in S^{\text{index}} \{ \text{ith\_element}(i^{\text{adj}}, S^{\text{macro}}), \dots \}$ ; % get the macroblocks given the set of indices
    return  $S^{\text{adj}}$ ;
}
```

Algorithm #4: Graph-based Clustering

% graph-based clustering is used for smaller problems

Algorithm graph-clustering

Input: 1. Set of points S^P where each point is a coordinate of the form (x, y) ;

2. length threshold τ ;

Output: set of clusters of edges S^{cluster} where each cluster itself is a set of edges;

Global: S^{valid} – set of valid edges with weight less than the threshold

% S^{mst} is a minimum spanning tree (mst) derived by greedy algorithm on S^P

% mst is modeled as a set of weighted edges of the form (src-node, dst-node, weight)

```
{   $S^{\text{mst}}$  = minimum_spanning_tree( $S^P$ ); % find the minimum spanning tree from the points
    % remove all edges having a weight greater than  $\tau$ 
     $S^{\text{valid}}$  = { }; % initialize the set of valid edges in minimum spanning tree
    while is_not_empty( $S^{\text{mst}}$ ) {
        next edge  $e$  = remove_first( $S^{\text{mst}}$ ); % extract the first edge from the set
        if ( $e$ .weight <  $\tau$ )  $S^{\text{valid}}$  =  $S^{\text{valid}}$  +  $e$ ; % if the weight < threshold  $\tau$ , it is a valid edge
    }
    while is_not_empty( $S^{\text{valid}}$ ) {
        edge  $e$  = remove_first( $S^{\text{valid}}$ ); % extract the shortest edge
         $S^{\text{cluNew}}$  = { $e$ }; % initialize the new cluster using the first extracted edge
         $S^{\text{fringe}}$  = { $\pi_1(e), \pi_2(e)$ }; % put both source and destination nodes in the fringe
        % findout connected graphs from  $S^{\text{fringe}}$ 
         $S^{\text{cluNew}}$  =  $S^{\text{cluNew}}$  + connected_graph( $S^{\text{cluNew}}$ , {},  $S^{\text{fringe}}$ ,  $S^{\text{valid}}$ );
         $S^{\text{cluster}}$  =  $S^{\text{cluster}}$  +  $S^{\text{cluNew}}$ ; % add the new cluster of edges to the set of clusters
        % remove the nodes in the newly found clusters from the set of remaining valid nodes
         $S^{\text{valid}}$  =  $S^{\text{valid}}$  -  $S^{\text{cluNew}}$ ; %  $S^{\text{valid}}$  on LHS is the set of remaining valid nodes
    }
    return  $S^{\text{cluster}}$ ;
}
```

Algorithm connected_graph % finds out the set of edges in a connected subgraph

Input: 1. set of edges in the cluster S^{cluNew} ; 2. Set of visited nodes S^{visited} ;

3. Unprocessed nodes S^{fringe} ; 4. Set of valid edges S^{valid} ;

Output: 1. Final new cluster S^{cluNew} ;

```
{  while is_not_empty( $S^{\text{fringe}}$ )
    { next_node  $n$  = remove_first( $S^{\text{fringe}}$ ); % remove the first node from the fringe
      if  $n \notin S^{\text{visited}}$  { % expand the node if it already not visited
           $S^{\text{visited}}$  =  $S^{\text{visited}}$  +  $n$ ; % update set of visited nodes by inserting current node  $n$ 
           $S^{\text{edgeN}}$  = incident_edges( $n$ ,  $S^{\text{valid}}$ ); % get all the incident edges on node  $n$ 
           $S^{\text{nodesN}}$  = nodes( $S^{\text{edgeN}}$ ); % get the nodes adjacent to the current node  $n$ 
           $S^{\text{cluNew}}$  =  $S^{\text{cluNew}}$   $\cup$   $S^{\text{edgeN}}$ ; % insert incident edges on node  $n$  in the new cluster
           $S^{\text{fringe}}$  =  $S^{\text{fringe}}$   $\cup$   $S^{\text{nodesN}}$ ; % increase the fringe by adding nodes adjacent to current fringe
           $S^{\text{valid}}$  =  $S^{\text{valid}}$  -  $S^{\text{edgeN}}$ ; % remove processed incident edges of  $n$  from the pool
      }
    }
    return  $S^{\text{cluNew}}$ ;
}
```

Algorithm minimum_spanning_tree**Input:** set of edges S^e defining a graph;**Output:** set of edges defining a minimum spanning tree;

```

{    $S^{cur} = \text{sort\_on\_weight}(S^e)$ ; % Ascending sort the set of edges by weight
     $S^{mst} = \{ \}$ ; % initialize the set of edges in minimum spanning tree to an empty set
    while (is_not_empty( $S^{cur}$ )){ % iteratively join two subgraph by the next shortest edge
        edge  $e = \text{remove\_first}(S^{cur})$ ; % remove the next shortest edge
        % find the set of edges in a subgraph connected to src-node
         $S^{sg1} = \text{connected\_graph}(\pi_1(e), S^{cur})$ ;
        % find the set of edges in a subgraph connected to dstn-node
         $S^{sg2} = \text{connected\_graph}(\pi_2(e), S^{cur})$ ;
         $S^{nodes1} = \text{nodes}(S^{sg1})$ ; % derive the set of nodes in the first subgraph
         $S^{nodes2} = \text{nodes}(S^{sg2})$ ; % derive the set of nodes in the second subgraph
        % if the intersection of two sets in empty subgraphs are dissociated and need to be connected
        if ( $S^{nodes1} \cap S^{nodes2} == \emptyset$ ){ % check the condition that nodes in two graphs do not share nodes
             $S^{joined} = S^{sg1} \cup S^{sg2} + e$ ; % join the two subgraphs by taking union of edge-sets + edge e
             $S^{mst} = S^{mst} \cup S^{joined}$ ; % put the derived edges in the set of edges of mst;
             $S^{cur} = S^{cur} - S^{joined}$ ; % remove separated subgraphs 1 and 2 from the available nodes
        }
    }
    return  $S^{mst}$ ;
}

```

Algorithm incident_edges**Input:** 1. node n ; 2. Set S^{edges} of available edges in the graph;**Output:** a set of incident edges $S^{incident}$ containing node n ;

```

 $S^{incident} = \{ \}$ ; % initialize the set of incident edges to an empty set
{ while is_not_empty( $S^{edges}$ ){ % process all the edges in the pool to check src-nodes or dst-nodes
     $e = \text{remove\_first}(S^{edges})$ ; % remove the first edge
    % add the edge to set of incident edges if node is a source or destination node
    if ( $n.\text{label} == \pi_1(e) \vee n.\text{label} == \pi_2(e)$ )  $S^{incident} = S^{incident} + e$ ;
}
return  $S^{incident}$ ;
}

```

Algorithm nodes**Input:** 1. set of edges S^{edges} ;**Output:** set of nodes S^{nodes} ;

```

{    $S^{nodes} = \{ \}$ ; % initialize the set of nodes to an empty set
    while (is_not_empty( $S^{edges}$ )){ % process all the edges
         $e = \text{remove\_first}(S^{edges})$ ; % remove the first edge from the set of edges
        % if the source-node or dst-node of the edge is not a member of set of nodes include it.
        if ( $\pi_1(e) \notin S^{nodes}$ )  $S^{nodes} = S^{nodes} + \pi_1(e)$ ;
        if ( $\pi_2(e) \notin S^{nodes}$ )  $S^{nodes} = S^{nodes} + \pi_2(e)$ ;
    }
    return  $S^{nodes}$ ;
}

```

Lectures 12, 15A HMM and FFN Algorithms

Algorithm # 1: HMM and Optimal Path

Algorithm HMM_path;

Input: 1. A set of states S^{States} ; 2. A set of emissions S^{emission} ;
3. A transition matrix $M^{\text{transition}} = S^{\text{States}} \times S^{\text{States}}$;
4. An emission matrix $M^{\text{emission}} = S^{\text{States}} \times S^{\text{emission}}$;
5. An initial state probability vector V^{initial} ;
6. A sequence of emissions $\text{Seq}^{\text{emission}}$;

Output: Most probable sequence of states Seq^{S} emitting Seq^{em} ;

```
{  read  $S^{\text{States}}$ ,  $S^{\text{emission}}$ ,  $M^{\text{tr}}$ ,  $M^{\text{emission}}$ ,  $V^{\text{initial}}$  from the file F;
   $N^{\text{States}} = \text{length}(S^{\text{States}})$ ; % derive the number of states in the proposed machine
   $N^{\text{emission}} = \text{length}(S^{\text{emission}})$ ; % compute the number of emissions given
  % use a variable get_path to exit when interacting with the user
  get_path = true; % initialize the variable get_path to start deriving the path
  while (get_path) { %keep deriving the path until get_path is true
    read( $\text{Seq}^{\text{em}}$ ) from the terminal; % read the next emission sequence
    if ( $\text{Seq}^{\text{emission}} = \text{"exit"}$ ) {get_path = false; exit}; % exit if the sequence is the string "exit"
    if (valid_emissions( $\text{Seq}^{\text{emission}}$ ,  $S^{\text{emission}}$ )) { % check for a valid sequence
      Transition-set  $S^{\text{transition}} = \{ \}$ ; % initialize the set of transitions to an empty set
       $N^{\text{seqEm}} = \text{length}(\text{Seq}^{\text{emission}})$ ; %compute the size of emission sequence
      i = 1; % initialize index I to 1 for iteratively processing emissions one by one
      while (i  $\leq$   $N^{\text{seqEm}}$ ) { % For each emission, compute the probable set of states that can emit it
        x = ith_element( $\text{Seq}^{\text{emission}}$ , i); % get the next emission from the sequence
         $S_i^{\text{probEm}} = \text{emission\_set}(x, M^{\text{emission}}, S^{\text{emission}}, N^{\text{states}})$ ; % compute the possible set emitting x
      }
      % Compute Cartesian product of possible emissions for each emission in the sequence
       $S^{\text{Cartesian}} = \text{Cartesian-product}(< S_1^{\text{probEm}}, \dots, S_{N_i}^{\text{probEm}} >)$ ;
       $S^{\text{Path}} = \{ \}$ ; % initialize set of probable paths to null set
       $\forall$  (pathSeq  $\in S^{\text{Cartesian}}$ ) {
        if (valid_transition(pathSeq,  $M^{\text{transition}}$ ,  $S^{\text{states}}$ ,  $V^{\text{initial}}$ ))  $S^{\text{Path}} = \text{pathSeq} + S^{\text{Path}}$ ;
      }
       $S^{\text{probSeq}} = < >$  % initialize the probability-score sequence to null sequence;
      maxSeq $^{\text{Emission}} = (\text{"", 0.0})$ ; % initialize the maximum probability
       $\forall$  path  $\in S^{\text{Path}}$  { % process each probable path and progressively find maximum probability
        probability prob = calculate_probability(path,  $\text{Seq}^{\text{emission}}$ ,  $S^{\text{states}}$ ,  $S^{\text{emission}}$ ,  $M^{\text{transition}}$ ,  $M^{\text{emission}}$ );
         $S^{\text{probSeq}} = (\text{path}, \text{prob}) + S^{\text{probSeq}}$ ; % insert the path sequence and the probability;
        print("next probable sequence is:", (path, prob));
        % compare with the current maximum probability sequence and update maxSeq
        if ( $\pi_2(\text{maxSeq}^{\text{emission}}) < \text{prob}$ ) maxSeq $^{\text{emission}} = (\text{path}, \text{prob})$ ;
      }
    }
  }
  return(maxSeq $^{\text{emission}}$ ); % return the sequence with maximum probability
}
```

Algorithm valid_emission**Input:** 1. The sequence $\text{Seq}^{\text{emission}}$; 2. Set of emissions S^{emission} ;**Output:** 2. Boolean value true or false;

% if any character of the emission sequence is missing from the set of emissions then it is invalid

{ $\exists \text{char} (\text{char} \in \text{Seq}^{\text{em}}) \wedge (\text{char} \notin S^{\text{emission}})$ **return** (false); **else return** (true);}**Algorithm valid_transition****Input:** 1. Possible transition sequence seq ;2. Transition matrix $M^{\text{transition}}$;3. Set of states S^{States} ;4. An initial state probability vector V^{initial} ;**Output:** Boolean value **true** or **false**;

```

{
  n = length(seq); % find the number of emissions
  m = 1; % initialize the index m to 1 for iteration over the sequence of emissions
  path_possible = true; % initialize the Boolean variable that is set if path is possible
  j = index(first(seq),  $S^{\text{States}}$ ); % extract the row number of the first state in the sequence
  % if the initial vector has a zero value then path is not possible and skip the path
  % test checks value smaller than epsilon because of floating point value of the probability
  if ( $|V^{\text{in}}[j]| < \epsilon$ ) path_possible = false; % check if the initial state probability is zero
  % check for every state in the possible path if probability is nonzero
  % a zero entry in the sequence makes it invalid path
  while((m < n)  $\wedge$  path_possible) % process all the states in the sequence one a time
    i = first(seq); j = second(seq); % pick the next two states in the sequence of states
    seq = rest(seq); % reduce the sequence by one element
    if (abs( $M^{\text{transition}}[i, j]$ ) <  $\epsilon$ ) path_possible = false;
    m = m + 1; % increment the index by one
  }
  if (path_possible) return(true); else return (false);
}

```

Algorithm emission_set % finds the set of states emitting the signal e**Input:** 1. Emission e ; 2. Emission matrix M^{emission} ; 3. Set of emissions S^{emission} ;4. no of states n^{states} ;**Output:** Possible emission set $S^{\text{emissionSet}}$;{j = index(e , S^{emission}); % find the index of the emission e

% For each element in the emission matrix row corresponding to the emission,

% collect the non-zero states iteratively

for ($i = 1$; $i \leq n$; $i++$) **if** ($|M^{\text{emission}}[i, j]| \geq \epsilon$) $S^{\text{emissionSet}} = i + S^{\text{emissionSet}}$;**return**($S^{\text{emissionSet}}$);

}

Algorithm calculate_probability

Input: 1. State sequence $\text{Seq}^{\text{States}}$;
2. Emission sequence $\text{Seq}^{\text{emission}}$;
3. set of states S^{States} ;
4. Set of emissions S^{emission} ;
5. Transition matrix $M^{\text{transition}}$;
6. Emission matrix M^{emission} ;

Output: probability $\text{prob}^{\text{path}}$;

```
{ nseq = length(Seq); % compute the length of the sequence
rowemission = index(first(SeqStates), SStates); % get the state that emits the first emission
colemission = index(first(Seqemission), Sem); % get the emission index for emission matrix
probpath = Vin[rowemission] × Memission[rowemission, colemission]; % initialize the cumulative probability
i = 2; % initialize the index to 2
while((i < n)) { % repeat updating transition to next state until sequence of emissions are over
    j = i + 1; % j is the index of the next state
    rowtransition = index(ith_element(SeqStates, i), Sstates); % find the state index for the transmission
    coltransition = index(ith_element(SeqStates, j), Sstates); % find the next state index for transmission
    rowemission = coltransition; % for the emission matrix get the row from the next state's index
    colemission = index(ith_element(Seqemission, j), Semission);
    probtransition = Mtransition[rowtransition, coltransition]; % retrieve probability of transmission
    probemission = Memission[rowemission, colemission] % retrieve the emission prob. of the next state
    probpath = probpath × probtransition × probemission; % update the intermediate pathprob
    i = i + 1; % increment the index to go to the next state
}
return(probpath) % return the final probability of the given path
}
```

Algorithm #2: Feed Forward Neural Network

Algorithm FeedForward_Neural_Network

Input: 1. Size of input vector s^{in} read from a file;

2. Size of output vector s^{out} read from a file;

3. A sequence of pair of training vectors $Seq^{training}$ read from a file;

4. Threshold for perceptron to fire τ^{fire} read from a file;

Global: 1. Sequence of trained matrices $S^{matrices} = \langle M_0, M_1, \dots, M_n \rangle$

Output: Sequence of trained matrices $S^{matrices}$

```
{ print("Give number of perceptrons in each hidden layer:"); phidden = read from terminal;
  print("Give the number of hidden layers:"); nhidden = read from terminal;
  print("Give the perceptron bias:"); bias = read from the terminal;
  print("Give cycles to be repeated:"); maxcycle = read from the terminal;
  print("Give the threshold for neural network adjustment:");  $\tau^{adj}$  = read from the terminal;
  print("Give the learning rate:");  $\eta$  = read from the terminal;
  Sinput =  $\Pi_1(Seq^{training})$ ; % extract sequence of input vectors by projecting first field for each training data
  Sout =  $\Pi_2(Seq^{training})$ ; % extract sequence of output vectors by projecting first field for each training data
  M0 = create_random_matrix( $s^{in}$ , phidden); % create a matrix for input layer to 1st hidden layer
  Mn = create_random_matrix(phidden, sout); % create a matrix for last hidden layer to output layer
  % create n random matrices one for each hidden layer with random weights
   $\forall i (1 \leq i \leq n - 1) M_i = \text{create\_random\_matrix}(p^{hidden}, p^{hidden})$ ; % initialize the weights
  bias_vector bvin = create_vector(bias, sin); % make a bias-vector for the input layer using a scalar value
  bias_vector_hidden_layer bvh = create_vector(bias, phidden); % create bias-vector for hidden layer
  bias_vector_output_layer bvout = create_vector(bias, sout); % create bias-vector for the output layer
  % keep adjusting the weights in the neural network either until the error is below a threshold  $\tau$ 
  % or the maximum number of iterative cycles are consumed
  index iteration = 1; Boolean adjusted = false;
  % repeatedly adjust the neural network until adjusted or runs out of maximum iteration Maxcycles
  while (iteration  $\leq$  maxCycles  $\wedge$   $\neg$  adjusted)
  {
     $\forall i (1 \leq i \leq s^{in}) \{V_0^{in}[i] = \text{ith\_element}(Seq^{in}, i)\}$ ; % initialize the vector  $V_0^{in}$ 
     $V_0^{in} = V_0^{in} + bv^{in}$ ; % calculate the input to the input layer and add bias b
     $V_0^{out} = \text{threshold\_fire}(V_0^{in})$ ; % output of the input layer after adding bias b and firing
    VC = process_hidden_layers(1, n); % run through the hidden layers to calculate output
    % calculate the error as Euclidean distance between the derived value VC and actual value
    error =  $\sqrt{\sum_{j=1}^{j=n} (\pi_j(V^C) - \pi_j(V_i^{out}))^2}$  % calculate the error from the target value

    % if the error is less than threshold then make adjusted as true to get out of the loop
    if (error >  $\tau^{adj}$ ) adjust_weight(Vactual, Vtarget); else adjusted = true;
  }
  if (adjusted) return(Smatrices); else iteration = iteration + 1; % go to the next iterative cycle
}
print('cycles over'); return(Smatrices); % run out of maximum cycles
}
```

Algorithm create_random_matrix % creates one layer of neural network using a random weight matrix

Input: 1. number of row n^{rows} ; 2. number of columns n^{col} ;

Output: 1. random matrix M;

```
{
    M = new_matrix( $n^{rows}$ ,  $n^{col}$ ); % create a new matrix
    Lower_bound lb = -1.0; upper_bound ub = 1.0; % initialize the bounds of random numbers
    % put random numbers in every element of the matrix
     $\forall i (1 \leq i \leq n^{rows}) \forall j (1 \leq j \leq n^{col})$  cell  $m_{ij} \in M = \text{random\_number}(lb, ub)$ ;
    return(M); % return the new matrix with random weights
}
```

Algorithm adjust_weight

Input: 1. Actual target sequence V^{out} ; 2. Computed sequence V^C ;

Global: 1. Hidden layer matrices set S^M ; 2. maximum count **max**; 3. error threshold τ^{adj} ;

4. Learning rate η ; 5. Hidden-layer vector size p^{hidden} ;

6. Number of hidden layers n^{hidden} ; 7. number of perceptrons in output layer p^{out} ;

```
{ current hidden layer  $I^{cur} = n^{hidden}$ ; % set the current hidden layer
     $E^{min} = \infty$ ; % initialize initial error to a very large number
    count = 1; % initialize the iterative index count
     $\epsilon = .00001$ ; % Any number less than  $\epsilon$  will be treated as zero entry in the matrix
    while ( $I^{cur} > 0 \wedge \text{count} \leq \text{max}$ )
    { % keep adjusting the weight starting from layer nearest to output layer until done
         $\Delta = V^{out} - V^C$ ; % get the difference vector between the computed value and actual value
         $S^{ind} = \{ \}$ ; % initialize the set of output values that differ in values
         $m = \text{length}(\Delta)$ ; % m is the number of fields in output value
        %  $\Delta$  is the vector of differences from the actual outcome
        %  $n^{hidden}$  is the total number of hidden layers
         $\forall i (1 \leq i \leq m)$  {if ( $\pi_i(\Delta) > \epsilon$ )  $S^{diff} = S^{diff} + (i, \pi_i(\Delta))$ ; } % remove nearly matching vector-bits;
        set of edges  $S^E = \text{get\_edges}(I, n^{hidden}, S^{diff})$ ; % find out the set of edges from previous layer
         $S^{Enew} = \text{modify\_edge\_weights}(S^E, I^{cur}, \Delta)$ ; % adjust weights of those edges to reduce error
         $S^{Mnew} = \text{insert\_modified\_edges}(S^{Enew}, I^{cur})$ ; % change the hidden layer I matrix  $S^M$ 
         $V^C = \text{process\_hidden\_layers}(I^{cur}, n^{hidden})$ ; % move forward again to calculate new value

        error =  $\sqrt{\sum_{j=1}^{p^{out}} (\pi_j(V^C) - \pi_j(V_i^{out}))^2}$ ; % calculate new error value

        % check if the matrix gives less error or error below the final required threshold
        % if the error is less than final threshold  $\tau$  then network is adjusted and exit
        % If the error is better than the previous error  $E^{min}$  then update  $E^{min}$ 
        if (error <  $E^{min}$ )  $\vee$  (error <  $\tau$ )
        { % update the matrix to new temporary matrix because it is better
             $S^M = S^{Mnew}$ ; if (error <  $\tau$ ) {adjusted = true; exit;} else  $E^{min} = \text{error}$ ;
        }

        count = count + 1; % go to the next iterative cycle
    }
    % if the network is trained or maximum count is reached then return
    if (adjusted or count  $\geq$  max) return;
    else  $I^{cur} = I^{cur} - 1$ ; % go to the previous hidden layer and adjust
}
```

Algorithm threshold_fire**Input:** 1. Cumulative vector V^{in} ;**Global:** Threshold of fire τ^{fire} ;**Output:** 1. Cumulative output vector V^{out} ;

```
{  $\forall i(1 \leq i \leq |V^{in}|)$  if  $(V^{in}[i] > \tau^{fire})$   $V^{out}[i] = \frac{1}{1 + e^{-V^{in}}}$  ; % use the exponential function for better resolution
    else  $V^{out}[i] = 0;$ }
```

Algorithm process_hidden_layers**Input:** 1. start hidden layer l^{start} ; 2. last hidden layer l^{end} ;**Global:** 1. bias vector bv^h ; 2. bias vector bv^{out} ; 3. sequence of matrix S^M ;**Output:** 1. Output vector V^C ;

```
{  $\forall k(l^{start} \leq k \leq l^{end})$ 
    { % process multiple hidden layers until last hidden layer
         $V_k^{in} = M_k \times V_{k-1}^{out} + bv^h$ ; % calculate the input to the first hidden layer
         $V_k^{out} = \text{threshold\_fire}(V_k^{in})$ ; % find the input value to be fed in the first hidden layer
    }
     $V_{n+1}^{in} = M_n \times V_n^{out} + bv^{out}$ ; % calculate the input to the output layer
     $V^C = \text{threshold\_fire}(V_{n+1}^{in})$ ; % output of the final output layer
}
```

Algorithm get_edges**Input:** 1. hidden layer l ;2. Current hidden layer $curl$;3. Set of edges S^{curE} contributing to difference between last hidden layer and output layer;**Global:** 1. hidden layer matrix-set S^M ;2. number of nodes in a hidden layer n^{hidden} ;3. number of hidden layers n^h **Output:** set of edges contributing edges S^{le} at layer l in the form (source-index, dest-index, weight);

```
{ % pull out the number of hidden layers
    if  $(curl = l)$  return  $S^{curE}$ ; % return the edges contributing to the changes in level  $l + 1$ 
    else { % go down to the proper layer one layer at a time to extract the set of affecting edges
        set of source nodes  $S^{src} = \Pi_1(S^{edges})$ ; % find out the source nodes from the previous hidden layer
        set of destination nodes  $S^{dst} = \Pi_2(S^{edges})$ ; % find dest. nodes in the current layer (output/hidden)
         $S^{curM} = \pi_{curl}(S^M)$ ; % get the matrix for the hidden layer  $h$ ;
         $S^{curE} = \{ \}$  % initialize the set of edges to an empty set
        % collect all the relevant edges affecting the next layer nodes
         $\forall r(1 \leq row\ r \in S^{src} \wedge r \leq n^{hidden}) \forall (col\ c \in S^{dst})$ 
        { % r denotes row and c denotes column
            weight =  $S^{curM}_{rc}$ ; % extract the weight from the hidden layer matrix  $S$ 
            if  $(|weight| > \epsilon)$   $S^{curE} = S^{curE} + (r, c, weight)$  % collect the non-zero weight edge
        }
         $curl = curl - 1$ ; % decrement the current hidden layer by 1
        get_edges( $l, curl, S^{curE}$ ); % call get_edges recursively
    }
}
```

Algorithm modify_edge_weights

Input: 1. set of edges $S^E = \{ (\text{matrix-level}, \text{row}, \text{column}, \text{value}), \dots \}$;
 2. hidden layer h ; % the specific hidden layer where to make changes
 3. value difference δ ; % the value difference between the derived value and actual value

Global: 1. learning rate η ; % learning rate tells how fast to change the weight

Output: 1. Set of edges $S^{\text{Enew}} = \{ (\text{row } r, \text{column } c, \text{weight } w), \dots \}$;
 $\{ S^{\text{Enew}} = \{ \};$ % initialize the set of updated weights
 $\forall ((\text{row}, \text{col}, \text{weight}) \in S^E) S^{\text{Enew}} = S^{\text{Enew}} + (\text{row}, \text{col}, \text{weight} \times (1 + \eta\delta));$ % update the weight
return S^{Enew} ; % return the set of new edges
 $\}$

Algorithm insert_modified_edges

Input: 1. list of modified edges S^{modE} ;
 2. hidden layer h where edges have to be modified;

Global: 1. number of nodes in a hidden layer n ;
 2. current hidden-layers matrix sequence S^M ;
 3. Number of hidden layers n^{hidden} ;

Output: updated hidden-layer matrix sequence S^{modM} ;
 $\{$ $S^{\text{modM}} = \{ \};$ % initialize the modified set of matrix sequence;
 $S^{\text{sortedE}} = \text{sort}(S^{\text{modE}})$ % sort on first dimension followed by second dimension
 % $\pi_i(S^M)$ is the matrix for i th hidden layer
 $\forall i (1 \leq i < h) S^{\text{modM}} = S^{\text{modM}} + \pi_i(S^M);$ % copy all matrices before the layer h
 % copy all modified weights in the corresponding matrices
 $\forall (\text{edge} \in S^{\text{modE}})$ **do**
 $\{$
 $\text{row } r = \pi_1(\text{edge});$ % row of the element to be updated in the hidden layer h
 $\text{col } c = \pi_2(\text{edge});$ % column of the element to be updated in the hidden layer h
 $\text{nw} = \pi_3(\text{edge});$ % new weight of the cell in the hidden layer h
 $S^{\text{modM}}[r, c] = \text{nw};$ % put the new value in the hidden layer matrix
 $\}$
 % copy all the matrices after the hidden layer h
 $\forall i (h + 1 \leq i \leq n^{\text{hidden}}) S^{\text{modM}} = S^{\text{modM}} + \pi_i(S^M)$ %
 $\}$